软　件　工　程

实验三：软件实现与构建

校园二手交易系统的编码实现、静态检查与仓库托管

课程名称　软件工程

项目名称　校园二手交易系统（CampusTrade）

学　　号　241220107

姓　　名　黄　毅

学　　院　计算机科学与技术学院

专　　业　计算机科学与技术

任课教师　王　豫

完成日期　2026 年 11 月

依据实验一 19 条需求与实验二 6 张 UML 图实现
Python 3，22 个源文件、3311 行，73 个单元测试
远程仓库 APY-tech/Software-Engineering

目录

1 实现概述 **1**

2 从 UML 图到代码 **1**

3 大模型辅助开发 **5**

4 代码风格与静态检查 **7**

5 编译与运行结果 **8**

6 Git 远程代码管理 **12**

1 实现概述

实验二给出了 10 个领域类、7 个商品状态、19 条需求的设计，本实验将其实现为可运行的程序。语言用 Python 3.13，只用标准库，界面为命令行菜单。选 Python 是因为系统难点在规则组合而非界面，写成测试逐条验证最省力；只用标准库可保证机房与自备机器结果一致。不做图形界面是权衡后的决定，界面层与业务层已完全分离，以后换 Tkinter 或 Web 只需替换 `cli` 目录。

代码分五层，依赖单向向下（表 1-1）。装配集中在 `context.py` 的 `build_application()`，依赖顺序写在一处。

表 1-1 代码分层与职责

层	文件	职责	行数
领域模型	`models.py`、`errors.py`、`state.py`	实体、枚举、异常层次、状态迁移表	513
基础设施	`storage.py`、`security.py`	内存表 + JSON 快照、口令散列与会话令牌	305
服务层	`services/`（9 个文件）	认证、商品、审核、交易、举报、统计、日志与通知	1343
界面层	`cli/shell.py`、`cli/tables.py`	菜单、输入校验、表格排版	602
装配与入口	`context.py`、`seed.py`、`demo.py`、`run.py`	依赖装配、演示数据、端到端脚本、参数解析	548
合计	22 个源文件	——	3311

贯穿实现的几条约定：

- 状态只有一处可写：迁移走 `apply_status()`，先查表再赋值并递增版本号，非法迁移抛 `Conflict`。
- 权限判断跟着服务走：方法首参为会话令牌，内部换成 `User` 再判角色与归属。
- 持久化用整库快照加原子替换：写临时文件后 `os.replace()`。
- 并发只在必要处加锁：`reserve()` 与 `cancel/confirm` 在临界区内完成查重—校验—写入。
- 异常分五类 `ValidationError`/`AuthError`/`PermissionDenied`/`NotFound`/`Conflict`，界面层只在一处捕获。

2 从 UML 图到代码

每种 UML 元素只对应一种写法，规则先定下，后面逐类翻译即可（表 2-1）。

表 2-1 UML 元素到 Python 构造的转换规则

UML 元素	代码构造	处理要点
类（实体）	`@dataclass`	字段照抄；主键统一暴露 `ident`
枚举	`class X(str, Enum)`	便于 JSON 落盘与按名字解析
关联（1—N）	外键字段 + 仓储查询	不挂对象引用，避免循环引用
组合（Item—ItemRevision）	独立表 + 按 `itemId` 查	待审内容能被单独丢弃
操作（方法）	服务层方法	跨聚合读写放服务层
状态图	迁移表 + `apply_status`	每条箭头一个表项
时序图	一个方法体 + 锁	`alt` 分支即 `if/else`

2.1 类图

领域模型（图 2-1）照实验二类图逐类翻译，10 个实体加 5 个枚举落在 `models.py`。实体只保留自身判断，跨实体协作放服务层。`ItemRevision` 是组合状态载体：已发布商品被修改时 `Item.status` 仍为已发布，新内容暂存于待审记录，审核通过后原子替换。`Reservation.requestId` 是幂等键。

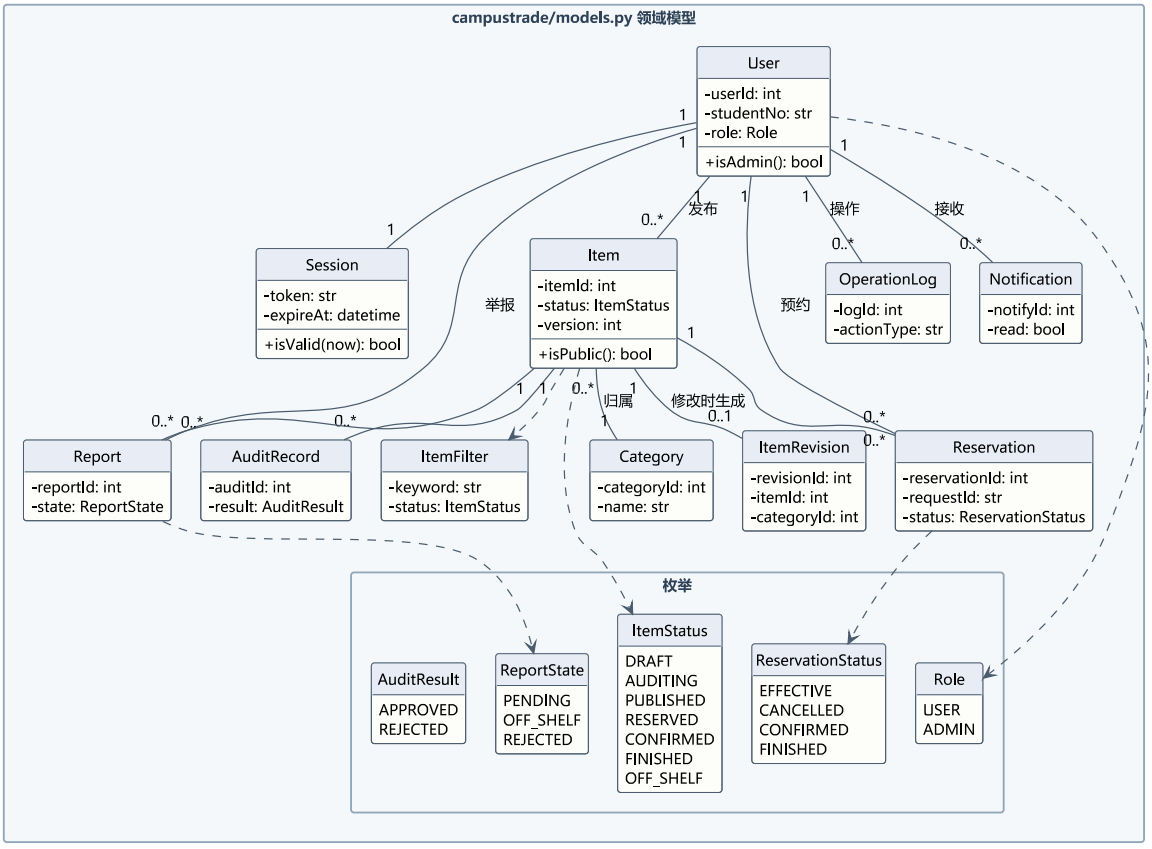


图 2-1 领域模型类图

服务层与仓储（图 2-2）是实现期新增：`ServiceBase` 收敛令牌换用户与落盘，六个业务服务继承它；`Store` 是仓储门面，`Table` 是泛型内存表。

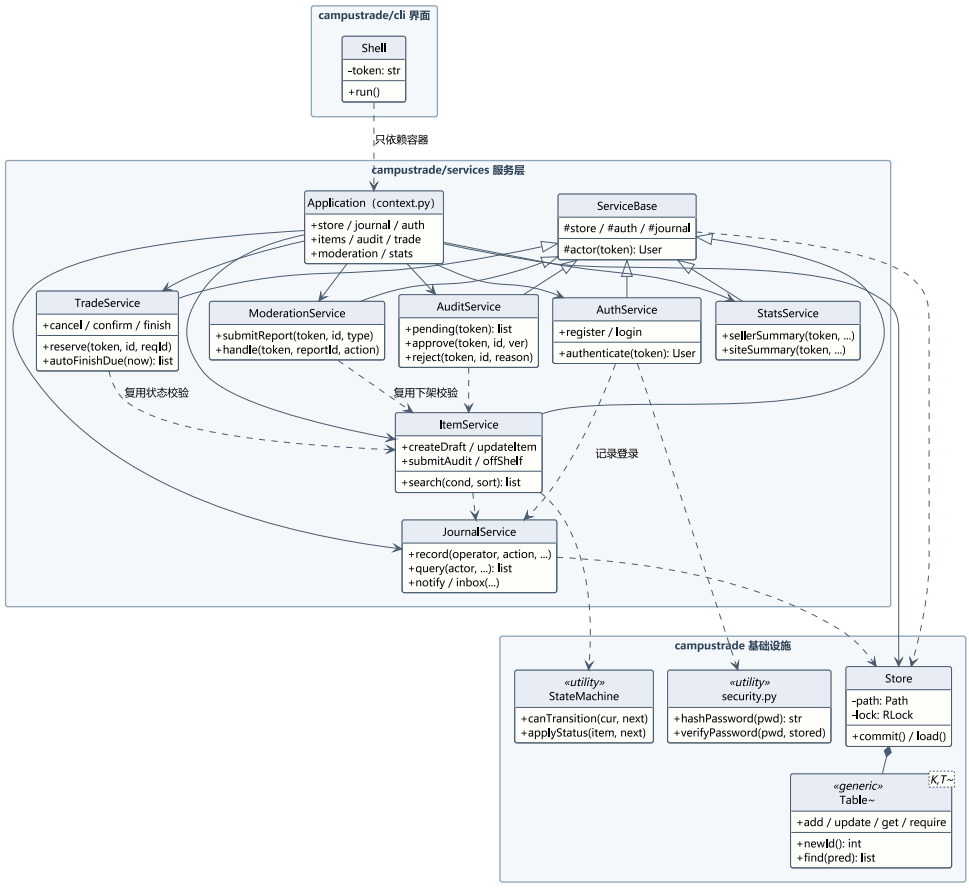


图 2-2　服务层与仓储类图

实现时对原设计改了三处：一是实体上的业务方法上移到服务层（方法需跨聚合读写，实体会被迫持有仓储引用，事务边界不清）；二是 `ItemRevision` 增加 `categoryId`（改分类也属修改商品信息，不带分类会导致审核后内容与分类不一致）；三是新增 `ServiceBase`、`Store`、`Table` 三个工程类，收敛重复代码。

2.2　状态图

7 个状态、9 条合法迁移写成 `state.py` 里的一个字典，`apply_status()` 先校验再赋值并递增 `version`（图 2-3、表 2-2）。非法迁移一律抛 `Conflict`。

2.3　时序图

预约流程按图 2-4逐条翻译，`alt` 分支对应 `raise`，`with store.lock` 对应临界区。查重放在最前，重发请求此时商品已是已预约，若先校验状态会把重发误判为冲突。

修改已发布商品（图 2-5）把待审内容与商品状态解耦：修改只写 `ItemRevision`，前台仍显示旧信息，审核期间买家照常预约；通过时原子替换全部字段，驳回则丢弃待审记录。

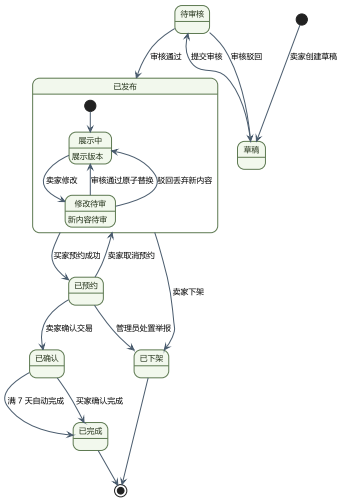


图 2-3　商品状态图

表 2-2　状态迁移与触发动作

迁移	触发方法	附加校验
草稿 → 待审核	`ItemService.submit_audit`	必填字段；仅本人
待审核 → 已发布	`AuditService.approve`	仅管理员；版本比对
待审核 → 草稿	`AuditService.reject`	必填驳回原因
已发布 → 已预约	`TradeService.reserve`	锁内查重；幂等键
已预约 → 已发布	`TradeService.cancel`	仅卖家
已预约 → 已确认	`TradeService.confirm`	仅卖家
已确认 → 已完成	`TradeService.finish`	仅预约买家；满 7 天自动完成
已发布 → 已下架	`ItemService.off_shelf`	仅本人或管理员
已下架 → 已发布	`ItemService.restore`	仅管理员

2.4　代码规模

共 22 个源文件、3311 行，去掉空行与注释后 **2761 行**；另有 8 个测试文件、73 个用例（表 2-3）。

表 2-3　源代码规模统计

分组	文件数	总行数	有效代码行	主要文件
领域模型	3	513	377	`models.py`（327）
基础设施	2	305	254	`storage.py`（260）
服务层	9	1343	1186	`item.py`（386）、`trade.py`（253）
界面层	3	602	500	`shell.py`（530）
装配与入口	5	548	444	`demo.py`（344）
小计	22	3311	**2761**	平均每文件 126 行
测试	8	794	577	73 个用例

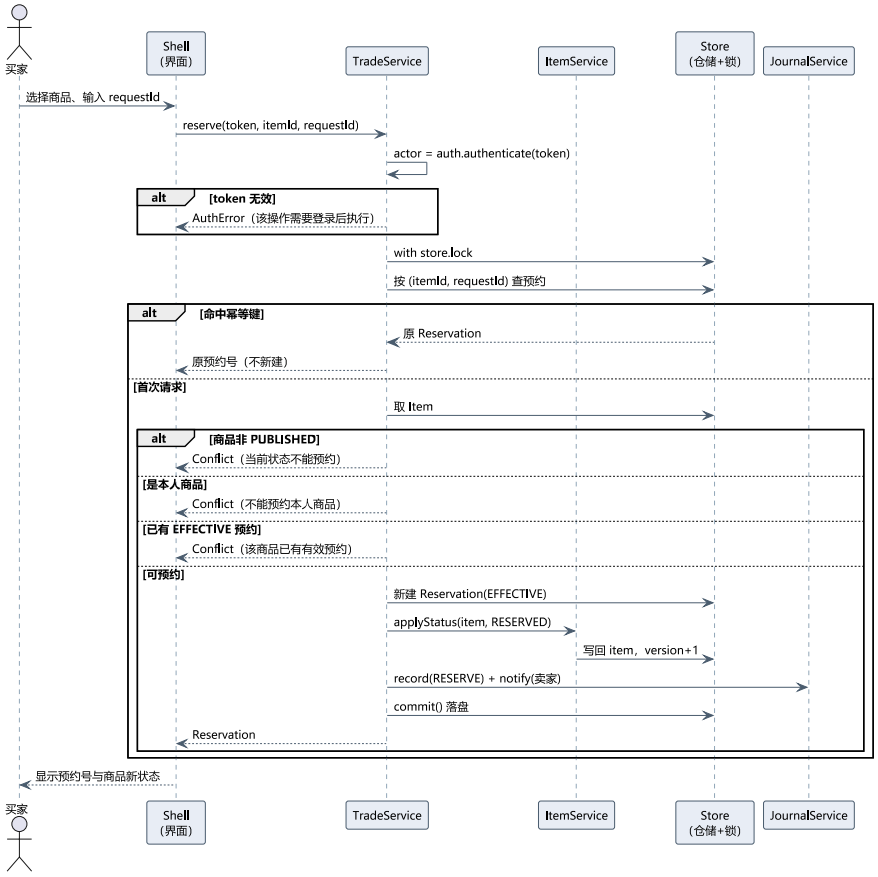


图 2-4　预约商品时序图

3　大模型辅助开发

3.1　使用方式

没有让模型一次把系统写出来，而是按模块切分、先规约后实现、每轮跑检查，共十轮：

- 一轮只喂一个模块：相关需求原文 + 类图片段 + 已定的接口签名。
- 先要存根与规约（方法签名、前置后置条件、会抛哪些异常），确认后再填实现。
- 每轮结束跑 `compileall/pylint/pytest`，把报错原文贴回下一轮。
- 产出逐行过一遍再入库。

十轮交互见表 3-1，完整提示词在仓库 `code/docs/prompts.md`。

3.2　模型承担的工作

- 模板化代码：dataclass 字段声明、异常类层次、`argparse` 参数定义。
- 把图翻成表：状态图的 9 条箭头、时序图的 `alt` 分支，逐条列出”箭头 → 触发

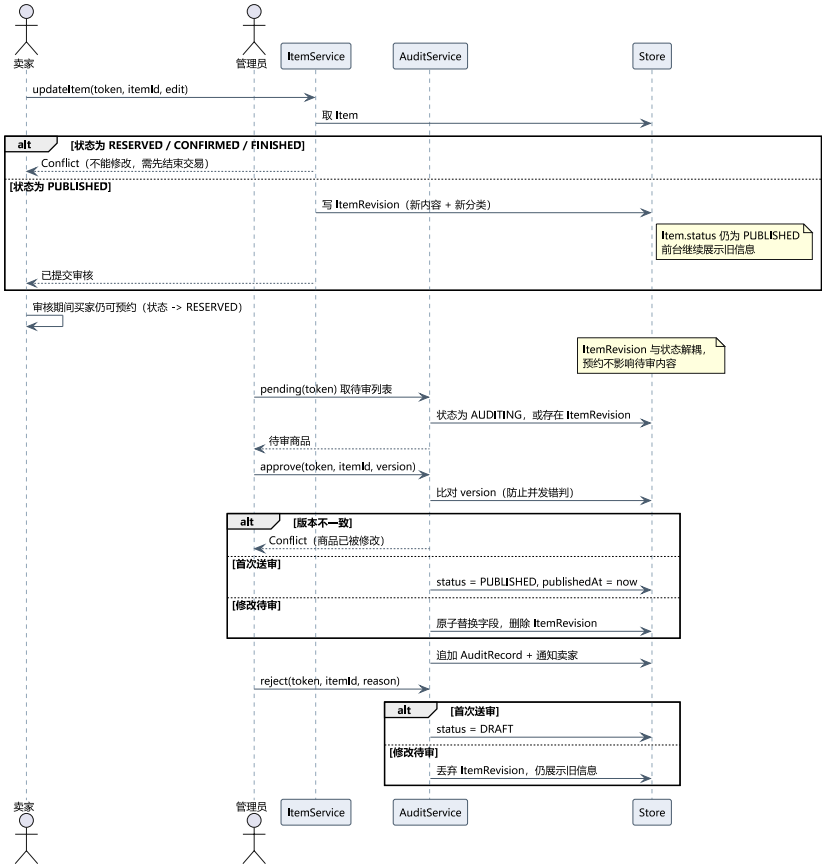


图 2-5　修改已发布商品与审核时序图

表 3-1　与大模型的十轮交互摘要

轮次	输入	模型产出	我的处理
R1	19 条需求 + 类图，只做数据模型	10 个 dataclass、5 个枚举、异常层次	采纳；补两个字段
R2	仓储要能落盘、支持泛型表	`Table`、`Store`、编解码	换掉 `asdict`，改原子写
R3	状态图 puml	迁移表 + `apply_status`	采纳；补中文状态名
R4	REQ-002/005/009/014/017	商品服务	**退回**：改为写待审记录
R5	REQ-003/019	审核服务与审核记录	采纳；补版本号比对
R6	REQ-001/006/007/008/015	交易服务	**退回**：改为锁内查重建单
R7	REQ-010/013/018	举报、日志、统计	采纳；处置复用商品下架
R8	全部服务	演示脚本、命令行菜单、表格	采纳结构；修 3 处残留
R9	pylint 27 条告警	逐条修复方案	采纳 25 条
R10	pytest 失败输出	定位” 完成只通知卖家”	采纳，改代码不改测试

方法 → 抛什么异常”。

- 按告警给修复方案：pylint 27 条告警贴回去，给出折行、改类型、多出口改单一出口等方案。

设计判断类的活它给不出可靠答案：锁加在哪里、要不要把方法从实体上移走、驳回后商品回不回草稿，都得自己定。

3.3 生成不符合预期时的处理

办法是造反例：每条它声称的规则，都在测试里造一个应该被拒绝的场景去撞，撞不穿才算数。下面四个缺陷都是撞出来的。

案例 1：审核漏掉” 审核期间被预约” 的状态。演示跑到第 7 节抛 `Conflict`，初稿只认待审核与已发布两种状态。REQ-017 写明审核期间买家仍可预约，商品会变成已预约。把可挂待审内容的状态抽成常量 `_REVISION_HOLDERS`，并补用例钉住。

```
# 初稿：只认待审核与已发布
if item.status is ItemStatus.AUDITING:
    apply_status(item, ItemStatus.PUBLISHED)
elif item.status is ItemStatus.PUBLISHED and revision is not None:
    self._apply_revision(item, revision, now)
else:
    raise Conflict("不在待审核状态")

# 修改后：可挂待审内容的状态抽成常量
_REVISION_HOLDERS = (ItemStatus.PUBLISHED, ItemStatus.RESERVED,
                     ItemStatus.CONFIRMED)
elif revision is not None and item.status in _REVISION_HOLDERS:
    self._apply_revision(item, revision, now)
```

案例 2：交易完成只通知卖家。REQ-015 要求” 自动完成并通知买家”，初稿只通知卖家。改 `_finish` 同时通知买卖双方，测试一行没动。

案例 3：必填校验放在建草稿时。初稿在 `create_draft` 就强制描述非空，写一半的草稿存不下来。改成两档：建草稿只查标题/价格/分类，提交审核时再查描述。

案例 4：用 `disable` 消告警。编解码函数被判 `too-many-return-statements`，模型建议加 `disable`。未接受，改成单一出口写法，告警因代码变简单而消失。

此外还退回了两种产出：把业务规则塞回实体方法的版本，以及” 先查再写” 不加锁的并发实现（两个线程可能都通过检查再各自写入，同一商品出现两条有效预约）。

4 代码风格与静态检查

用 `pylint` 4.1.3 按 PEP 8（行宽 79）检查。只关掉三类与作业无关的检查：`duplicate-code`（dataclass 声明必然相似）、`too-few-public-methods`（实体只

有数据）、`too-many-lines`；参数上限放宽到 6、属性上限 14（`Item` 有 13 个字段）。

初稿跑出 **9.85/10**，27 条告警全是格式与写法问题，无逻辑错误（表 4-1）。

表 4-1 初稿 27 条告警的分布

告警	条数	典型位置	处理方式
`line-too-long` C0301	22	6 个文件，最长 83 字符	按语义折行
`trailing-newlines` C0305	2	`item.py`、`moderation.py`	删末尾空行
`unidiomatic-typecheck` C0123	1	`storage.py`	`type(None)` 改 `types.NoneType`
`too-many-return-statements` R0911	1	`storage.py`	重写为单一出口
`unused-import` W0611	1	`auth.py`	删除未用导入

行宽占 22 条，按语义折行而非删空格：

```
# 修复前（83 字符）
self._journal.record(actor.user_id, action, item.item_id, True, item.title)

# 修复后：参数对齐到左括号后
self._journal.record(
    actor.user_id, action, item.item_id, True, item.title)
```

纳入测试代码后复查 **10.00/10**：

```
--------------------------------------------------------------------
Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)
```

测试目录有三处文件级 `disable`（夹具互相引用、用例名即文档、铺数据夹具），均写明理由；源码目录一条 `disable` 都没有。

5 编译与运行结果

运行环境为 Windows 11 + Python 3.13.14（仅标准库），检查工具 pylint 4.1.3、pytest 9.2.0，报告用 TeX Live 2025 的 XeLaTeX 排版。

5.1 编译检查

Python 没有独立编译步骤，用标准库 `compileall` 对全部源码做字节码编译，可暴露语法与导入错误：

```
$ python -m compileall -q campustrade run.py tests
$ echo $?
0
```

退出码为 0，22 个源文件与 8 个测试文件均通过检查。

5.2 单元测试

```
$ python -m pytest -q
```

```
.....................................................................  [ 98%]
.                                                                      [100%]
73 passed in 15.82s
```

测试按模块分文件（表 5-1），每条规则都配一个” 应该被拒绝” 的用例。

表 5-1 测试文件与覆盖的需求

测试文件	用例数	覆盖内容
test_auth.py	12	注册、登录、会话过期、停用账号、改密码（REQ-004/012/016）
test_item.py	15	草稿、查询过滤排序、必填校验、下架恢复、分类保护（REQ-002/005/009/014）
test_audit.py	10	通过/驳回、待审内容原子替换、版本冲突、审核历史（REQ-003/017/019）
test_trade.py	13	幂等重发、唯一有效预约、取消确认完成、超时自动完成、并发竞争（REQ-001/006/007/008/015）
test_moderation.py	8	举报去重、处置下架/驳回、重复处理（REQ-010/014）
test_storage.py	10	JSON 快照往返、时间范围统计、日志权限、损坏文件处理（REQ-013/018）
test_cli.py	5	登录、搜索、预约、非法选项、管理员菜单的界面冒烟
合计	73	——

5.3 端到端演示

python run.py --demo 在内存库上按需求编号跑 11 节场景。下面保留正文点名的五节，其余六节省略（✓ 成功、× 被拒绝）：

```
=== 1. 注册与登录（REQ-004） ==============================
  × 重复学号注册 -> 已拒绝：学号 241220101 已被注册
  × 密码错误登录 -> 已拒绝：学号或密码错误
  ✓ 登录态有效：241220101，角色 USER
  × 未登录直接预约 -> 已拒绝：该操作需要登录后执行

  ……（本节略）
  ……（本节略）
=== 4. 预约的幂等与唯一性（REQ-001） ======================
  ✓ 买家预约成功，预约号 1，商品状态 已预约
  ✓ 同一 requestId 重发返回原记录：预约号 1（共 1 条预约记录）
  × 另一买家再预约 -> 已拒绝：商品当前为已预约，不能预约
  × 卖家预约本人商品 -> 已拒绝：不能预约本人发布的商品
  × 缺少 requestId -> 已拒绝：缺少幂等键 requestId

  ……（本节略）
  ……（本节略）
=== 7. 修改已发布商品（REQ-017） ==========================
  ✓ 修改已发布商品后，前台仍显示《罗技 M590 静音鼠标》/89.00 元，状态 已发布
```

```
  √ 新内容暂存于待审记录 #1，提交时间 21:28:23
  √ 该商品进入待审核列表：True
  √ 修改审核期间买家仍可预约：预约号 3，前台仍是旧标题《罗技 M590 静音鼠标》
  √ 审核通过后原子更新为 79.00 元，待审记录已清除：True
  √ 驳回修改后仍为已发布，显示原价 45.00 元，待审内容已丢弃：True

  ……（本节略）
=== 9. 并发预约（REQ-001 竞争场景） ========================
  √ 4 个买家同时预约，结果：['成功', '失败（CONFLICT）', '失败（CONFLICT）',
    '失败（CONFLICT）']
  √ 商品最终状态 已预约，有效预约 1 条

=== 10. 超时自动完成（REQ-015） ============================
  √ 把确认时间回拨 8 天，模拟时间流逝
  √ 自动完成 1 笔，商品状态 已完成
  √ 卖家共收到 16 条通知，最新一条：《护眼台灯》交易已完成，金额 35.00 元
```

几个关键结果：第 4 节同一 `requestId` 重发返回原预约号、记录仍只 1 条；第 7 节修改后前台仍显示旧标题旧价格，期间买家仍能预约，审核通过后原子替换；第 9 节 4 个买家并发预约只有 1 个成功；第 10 节回拨 8 天触发自动完成，买卖双方均收到通知。

5.4 交互式运行

`python run.py --seed` 进入菜单界面，下面是真实会话（省去重复的菜单与输入提示，→ 后为执行结果）：管理员审核通过待审商品，买家搜索并预约，卖家确认交易，买家确认完成，最后卖家查看统计。

```
已初始化演示数据：管理员 900000001，密码 ********
==================================================================
                    校园二手交易系统 • 命令行版
==================================================================

  → 登录成功。

  当前身份：900000001（管理员）
  编号  标题                    状态    版本  待审修改
  ----  ----------------------  ------  ----  --------
     3  考研英语真题 2010-2024  待审核     2  无

  当前身份：900000001（管理员）
  → 审核通过，商品状态：已发布

  当前身份：900000001（管理员）
  → 登录成功。

  当前身份：241220102（买家/卖家）
  → 命中 3 件
  编号  标题                     价格  状态    发布
  ----  ----------------------  -----  ------  -----------
     3  考研英语真题 2010-2024  45.00  已发布  10-31 21:28
     2  罗技 M590 静音鼠标      89.00  已发布  10-31 21:28
     1  高等数学 第七版 上册    18.00  已发布  10-31 21:28
```

```
当前身份：241220102（买家/卖家）
→ 预约成功：预约号 1，请求号 req-cli-a

当前身份：241220102（买家/卖家）
预约号  商品  买家  状态  异议
------  ----  ----  ----  ----
     1     3     3  有效  否

当前身份：241220102（买家/卖家）
→ 登录成功。

当前身份：241220101（买家/卖家）
编号  标题                      价格  状态    发布
----  ----------------------  -----  ------  -----------
   3  考研英语真题 2010-2024  45.00  已预约  10-31 21:28
   2  罗技 M590 静音鼠标      89.00  已发布  10-31 21:28
   1  高等数学 第七版 上册    18.00  已发布  10-31 21:28
   4  待补图的书架            60.00  草稿    -

当前身份：241220101（买家/卖家）
→ 已确认交易，请与买家线下交付。

当前身份：241220101（买家/卖家）
→ 登录成功。

当前身份：241220102（买家/卖家）
→ 交易已完成。

当前身份：241220102（买家/卖家）
→ 登录成功。

当前身份：241220101（买家/卖家）
→ 发布 4 件，预约 1 次，完成 1 笔，交易额 45.00 元

当前身份：241220101（买家/卖家）
```

权限随身份变化：管理员菜单多出 21–30 项（审核、举报、统计、日志、分类），普通用户只有 1–15 项。末行统计” 发布 4 件，预约 1 次，完成 1 笔，交易额 45.00 元” 与前面操作对得上。

5.5 未覆盖的部分

- REQ-011 性能指标未实测：单机内存表的并发模型与 Web 服务不同，留待实验四做基准。
- REQ-012 的 HTTPS 在桌面程序里无对应物;密码加密存储部分已实现（PBKDF2-HMAC-SHA256，12 万轮，随机盐）。
- 图形界面未落地，实验二的 10 张界面稿仍停留在原型，控制台能覆盖全部用例但演示效果差。

6 Git 远程代码管理

本地仓库在 repo/，远程托管在 GitHub：

```
$ git remote -v
origin  https://github.com/APY-tech/Software-Engineering.git (fetch)
origin  https://github.com/APY-tech/Software-Engineering.git (push)
```

主分支 main，三次实验的产物在同一仓库里，靠目录与提交说明区分。本轮按”一类产物一次提交” 拆开（表 6-1）。

表 6-1 本次实验的主要提交

提交	文件数	内容
a66e16a	34	22 个源文件、8 个测试文件、配置与说明
31056f2	8	pylint 前后对比、测试与演示输出、提示词记录
ad62b25	12	类图、服务类图、两张时序图的 puml 与 png
e2e555f	20	报告 LaTeX 源、插图与编译产物

仓库提交历史：

```
=== git log (main) ===
9745ea9 实验三 代码留档：git 记录补全到 HEAD，附实验三提交明细
75bb7a1 实验三 代码留档：补充 git 提交历史与远程地址记录
e2e555f 实验三 报告：实现与构建报告 26 页，含真实检查与运行输出
ad62b25 实验三 UML：按实现修订类图，新增服务类图与两张时序图
31056f2 实验三 代码留档：pylint 前后对比、测试与演示输出、提示词记录
a66e16a 实验三 代码：19 条需求的 Python 分层实现，22 个文件
09a2689 实验二 报告修订：表4-1 每条记录显式给出优缺点，表4-2 编号与列宽
cb3bdc6 新增 README：仓库结构、重新生成方式与提交约定
2b79a57 实验二 报告：5 章 17 页，新增 AI 工作流与 git 追踪两节
a16c181 回改实验一：REQ-017 驳回后状态、REQ-001 幂等键、REQ-009 模块归属
961f4e0 实验二 UI：10 个界面稿与可交互原型源文件
76f4651 实验二 UML：六张图的 PlantUML 源码与渲染图
9745ea9 实验三 代码留档：git 记录补全到 HEAD，附实验三提交明细
75bb7a1 实验三 代码留档：补充 git 提交历史与远程地址记录
e2e555f 实验三 报告：实现与构建报告 26 页，含真实检查与运行输出
ad62b25 实验三 UML：按实现修订类图，新增服务类图与两张时序图
31056f2 实验三 代码留档：pylint 前后对比、测试与演示输出、提示词记录
origin  https://github.com/APY-tech/Software-Engineering.git (fetch)
origin  https://github.com/APY-tech/Software-Engineering.git (push)
## main...origin/main
```

.gitignore 排除 Python 缓存（__pycache__/、.pytest_cache/）、运行时数据（code/data/，可用 --seed 重建）与 LaTeX 中间产物；源码、UML、测试、检查记录、报告均入库。